1. Convex Hull - points with interior elements.

Given points

$A = (1,1)$

$B = (2,5)$

$C = (5,4)$

$D = (6,1)$

$E = (3,3)$

$F = (4,2)$

1. problem interpretation

1. Consider every pair of points.
2. Draw the line joining them
3. if yes, that line segment is a convex hull edge
4. otherwise, reject it

outer boundary consists of edges that separate all other points to one side

E(3,3)

F(4,2)

Hence they are expected to be interior points.

3. Total line pairs.

$$\binom{6}{2} = 15$$

Pair	Hull Edge	Reason
AB	✓	All remaining points lie on one side
AC	✗	points occur on both sides.
AD	✗	points on both sides
AE	✗	Both sides occupied
BC	✓	All other points below the line
CD	✓	All remaining points on one side.
CE	✗	Interior connection
DE	✗	Both sides occupied
EF	✗	Internal line

4. Validation of Hull Edge.

Edge AB : (1,1) - (2,5)

- C lies on same side
- D lies on same side
- E lies on same side
- F lies on same side

Therefore,

AB is a hull edge.

5. Convex Hull

The convex hull consists of four outer vertices

$(1,1) \rightarrow (2,5) \rightarrow (5,4) \rightarrow (6,1) \rightarrow (1,1)$

Interior points

- $(3,3)$
- $(4,2)$

6. Number of comparisons.

Number of candidate edges $\binom{6}{2} = 15$

Remaining points checked $6-2=4$

Total orientation side comparisons :

$15 \times 4 = 60$

Total comparisons $= 60$

8. Time Complexity.

For n points.

- Number of pairs :

$$\binom{n}{2} = O(n^2)$$

- For each pair, check the remaining $n-2$ points.

$O(n)$

Time complexity : $O(n^3)$

Space complexity : $O(1)$ (excluding storage of the inputs and outputs)

9. Computational Geometry Interpretation.

- The brute force convex hull algorithm identifies the outer boundary by testing every possible edge.
- An edge belongs to the hull only if all other points lie on the same side of its supporting line.
- Interior points such as (3,3) and (4,2) fail this condition and are excluded.

Although straightforward and easy to understand, the algorithm performs many redundant checks; resulting in $O(n^3)$ time complexity. for large datasets, more efficient algorithms such as Graham or Jarvis march are preferred because they [reduce] the computational cost.